

# TECNOLÓGICO DE MONTERREY

## Estructuras de Datos y Algoritmos Fundamentales

### *Evidencias de Pruebas de la Evidencia 1*

Nombre: Daniela Chávez Ibarra

Matrícula: A00843645

## 1. Plan de Pruebas

### Objetivos de la Validación

El objetivo principal del plan de pruebas es garantizar que la aplicación procese, ordene y busque de manera correcta y eficiente dentro de archivos de registros (logs) de eventos, asegurando la integridad de los datos y el manejo adecuado de errores.

### ¿Qué se validó y por qué?

**Algoritmos de Ordenamiento (7 Algoritmos x 2 Archivos):** Se validaron las 14 combinaciones posibles para comparar el rendimiento de tiempo de ejecución entre un archivo completamente desordenado (log607-1.txt) y uno casi ordenado (log607-2.txt), contrastando los resultados empíricos contra la complejidad teórica ( $O(n^2)$  vs  $O(n \log n)$ ).

**Búsqueda Binaria por Rango (lowerBound y upperBound):** Se validó que la selección del subrango de fechas funcione en tiempo logarítmico  $O(\log n)$  y que extraiga exactamente los registros del intervalo solicitado.

### Casos de Borde y Resiliencia:

**Timestamps duplicados:** Para asegurar que en log607-2.txt no se omitan ni se dupliquen registros cuya fecha coincida exactamente en los límites del rango.

**Rango vacío:** Para comprobar que el programa maneje con gracia búsquedas sin coincidencias.

**Bordes de la lista:** Validación de la búsqueda en las fechas extremas (primer y último registro).

**Entradas inválidas:** Para asegurar que el programa no colapse ante entradas incorrectas del usuario en la consola.

## 2. Resultados de Tiempos de Ordenamiento (Matriz 7 x 2)

A continuación, se muestra la tabla comparativa de tiempos de ejecución de cada algoritmo sobre ambos archivos de prueba:

| Número de prueba | Algoritmo      | Complejidad Teórica (Mejor / Peor) | log607-1.txt (Desordenado) | log607-2.txt (Casi Ordenado) | Coincide con Predicción |
|------------------|----------------|------------------------------------|----------------------------|------------------------------|-------------------------|
| 1 y 2            | Swap Sort      | $O(n^2)$ / $O(n^2)$                | 5786 ms                    | 547 ms                       | Sí                      |
| 3 y 4            | Bubble Sort    | $O(n)$ / $O(n^2)$                  | 5719 ms                    | 148 ms                       | No                      |
| 5 y 6            | Selection Sort | $O(n^2)$ / $O(n^2)$                | 434 ms                     | 535 ms                       | Sí                      |
| 7 y 8            | Insertion Sort | $O(n)$ / $O(n^2)$                  | 5649 ms                    | 139 ms                       | Sí                      |
| 9 y 10           | Merge Sort     | $O(n \log n)$ / $O(n \log n)$      | 51 ms                      | 49 ms                        | Sí                      |
| 11 y 12          | Quick Sort     | $O(n \log n)$ / $O(n^2)$           | 21 ms                      | 94 ms                        | No                      |
| 13 y 14          | Shell Sort     | $O(n \log n)$ / $O(n^2)$           | 37 ms                      | 30 ms                        | Sí                      |

## Evidencia de cada prueba

|                |                                                                                                                                                                                                                                                                                                            |  |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| Swap Sort      | <b>Evidencia Prueba #1 – desordenado</b>                                                                                                                                                                                                                                                                   |  |
|                | Ejecutando Swap Sort...                                                                                                                                                                                                                                                                                    |  |
|                | <pre> RESUMEN ..... Algoritmo elegido:      Swap Sort Archivo utilizado:      log607-1.txt (desordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    5786 ms Complejidad Teorica:    Mejor caso: O(n^2)   Peor caso: O(n^2) Comparaciones:          23239153   Swaps: 11573512 </pre>   |  |
| Swap Sort      | <b>Evidencia de Prueba #2 – casi ordenado</b>                                                                                                                                                                                                                                                              |  |
|                | Ejecutando Swap Sort...                                                                                                                                                                                                                                                                                    |  |
|                | <pre> RESUMEN ..... Algoritmo elegido:      Swap Sort Archivo utilizado:      log607-2.txt (casi ordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    547 ms Complejidad Teorica:    Mejor caso: O(n^2)   Peor caso: O(n^2) Comparaciones:          23239153   Swaps: 345176 </pre>    |  |
| Bubble Sort    | <b>Evidencia de Prueba #3 – desordenado</b>                                                                                                                                                                                                                                                                |  |
|                | Ejecutando Bubble Sort...                                                                                                                                                                                                                                                                                  |  |
|                | <pre> RESUMEN ..... Algoritmo elegido:      Bubble Sort Archivo utilizado:      log607-1.txt (desordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    5719 ms Complejidad Teorica:    Mejor caso: O(n)   Peor caso: O(n^2) Comparaciones:          23221575   Swaps: 11596219 </pre>   |  |
| Bubble Sort    | <b>Evidencia de Prueba #4 – casi ordenado</b>                                                                                                                                                                                                                                                              |  |
|                | Ejecutando Bubble Sort...                                                                                                                                                                                                                                                                                  |  |
|                | <pre> RESUMEN ..... Algoritmo elegido:      Bubble Sort Archivo utilizado:      log607-2.txt (casi ordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    148 ms Complejidad Teorica:    Mejor caso: O(n)   Peor caso: O(n^2) Comparaciones:          757288   Swaps: 345957 </pre>      |  |
| Selection Sort | <b>Evidencia de Prueba #5 – desordenado</b>                                                                                                                                                                                                                                                                |  |
|                | Ejecutando Selection Sort...                                                                                                                                                                                                                                                                               |  |
|                | <pre> RESUMEN ..... Algoritmo elegido:      Selection Sort Archivo utilizado:      log607-1.txt (desordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    434 ms Complejidad Teorica:    Mejor caso: O(n^2)   Peor caso: O(n^2) Comparaciones:          23239153   Swaps: 6808 </pre>   |  |
| Selection Sort | <b>Evidencia de Prueba #6 – casi ordenado</b>                                                                                                                                                                                                                                                              |  |
|                | Ejecutando Selection Sort...                                                                                                                                                                                                                                                                               |  |
|                | <pre> RESUMEN ..... Algoritmo elegido:      Selection Sort Archivo utilizado:      log607-2.txt (casi ordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    535 ms Complejidad Teorica:    Mejor caso: O(n^2)   Peor caso: O(n^2) Comparaciones:          23239153   Swaps: 6705 </pre> |  |
| Insertion Sort | <b>Evidencia de Prueba #7 – desordenado</b>                                                                                                                                                                                                                                                                |  |

|                   |                                                                                                                                                                                                                                                                                                                                                 |  |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
|                   | <p>Ejecutando Insertion Sort...</p> <pre> RESUMEN ..... Algoritmo elegido:      Insertion Sort Archivo utilizado:      log607-1.txt (desordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    5649 ms Complejidad Teorica:    Mejor caso: O(n)   Peor caso: O(n^2) Comparaciones:          11603034   Swaps: 11596219 </pre> |  |
|                   | <b>Evidencia de Prueba #8 – casi ordenado</b>                                                                                                                                                                                                                                                                                                   |  |
|                   | <p>Ejecutando Insertion Sort...</p> <pre> RESUMEN ..... Algoritmo elegido:      Insertion Sort Archivo utilizado:      log607-2.txt (casi ordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    139 ms Complejidad Teorica:    Mejor caso: O(n)   Peor caso: O(n^2) Comparaciones:          352772   Swaps: 345957 </pre>    |  |
| <b>Merge Sort</b> | <b>Evidencia de Prueba #9 – desordenado</b>                                                                                                                                                                                                                                                                                                     |  |
|                   | <p>Ejecutando Merge Sort...</p> <pre> RESUMEN ..... Algoritmo elegido:      Merge Sort Archivo utilizado:      log607-1.txt (desordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    51 ms Complejidad Teorica:    Mejor caso: O(n log n)   Peor caso: O(n log n) </pre>                                                    |  |
|                   | <b>Evidencia de Prueba #10 – casi ordenado</b>                                                                                                                                                                                                                                                                                                  |  |
|                   | <p>Ejecutando Merge Sort...</p> <pre> RESUMEN ..... Algoritmo elegido:      Merge Sort Archivo utilizado:      log607-2.txt (casi ordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    49 ms Complejidad Teorica:    Mejor caso: O(n log n)   Peor caso: O(n log n) </pre>                                                  |  |
| <b>Quick Sort</b> | <b>Evidencia de Prueba #11 – desordenado</b>                                                                                                                                                                                                                                                                                                    |  |
|                   | <p>Ejecutando Quick Sort...</p> <pre> RESUMEN ..... Algoritmo elegido:      Quick Sort Archivo utilizado:      log607-1.txt (desordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    21 ms Complejidad Teorica:    Mejor caso: O(n log n)   Peor caso: O(n^2) </pre>                                                        |  |
|                   | <b>Evidencia de Prueba #12 – casi ordenado</b>                                                                                                                                                                                                                                                                                                  |  |
|                   | <p>Ejecutando Quick Sort...</p> <pre> RESUMEN ..... Algoritmo elegido:      Quick Sort Archivo utilizado:      log607-2.txt (casi ordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    94 ms Complejidad Teorica:    Mejor caso: O(n log n)   Peor caso: O(n^2) </pre>                                                      |  |
| <b>Shell Sort</b> | <b>Evidencia de Prueba #13 – desordenado</b>                                                                                                                                                                                                                                                                                                    |  |
|                   | <p>Ejecutando Shell Sort...</p> <pre> RESUMEN ..... Algoritmo elegido:      Shell Sort Archivo utilizado:      log607-1.txt (desordenado) Tamano de datos:        6818 registros Tiempo de ejecucion:    37 ms Complejidad Teorica:    Mejor caso: O(n log n)   Peor caso: O(n^2) </pre>                                                        |  |
|                   | <b>Evidencia de Prueba #14 – casi ordenado</b>                                                                                                                                                                                                                                                                                                  |  |

|  |                                                                                                                                                                                                                                                                                                    |  |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
|  | Ejecutando Shell Sort...                                                                                                                                                                                                                                                                           |  |
|  | <p>RESUMEN</p> <p>.....</p> <p>Algoritmo elegido: Shell Sort</p> <p>Archivo utilizado: log607-2.txt (casi ordenado)</p> <p>Tamaño de datos: 6818 registros</p> <p>Tiempo de ejecución: 30 ms</p> <p>Complejidad Teórica: Mejor caso: <math>O(n \log n)</math>   Peor caso: <math>O(n^2)</math></p> |  |

### 3. Casos de prueba ejecutados

A continuación, se observan los casos de prueba requeridos en las instrucciones de Canvas.

#### Caso 1: Manejo de Entradas Inválidas en Consola

**Descripción:** Se ingresó “Abc” en la sección del mes y una entrada de caracteres en la sección de Día donde le corresponde datos de tipo entero para comprobar la recuperación de la entrada (`cin.clear()` y `cin.ignore()`).

**Resultado:** El sistema mostró una advertencia clara, limpió el búfer y solicitó nuevamente la entrada sin colapsar ni entrar en bucle infinito. Volvió a comenzar desde la fase de pedir al usuario que seleccione el archivo que quiere utilizar.

```

└─┐Deseas realizar una búsqueda por rango de fechas en estos datos ordenados? (s/n): s
-----
Busqueda por rango de fecha
-----
--- Ingrese la Fecha INICIAL ---
Mes (ej. Sep): Abc
Dia (ej. 29): ad
Año (ej. 2024): Hora (HH:MM:SS, ej. 14:00:00):
--- Ingrese la Fecha FINAL ---
Mes (ej. Oct): Dia (ej. 05): Año (ej. 2024): Hora (HH:MM:SS, ej. 18:00:00):
No se encontraron registros dentro del rango de fechas especificado.

└─┐Desea realizar otra corrida para comparar? (s/n): EVIDENCIA 1
-----
1. Cargar log607-1.txt (desordenado)
2. Cargar log607-2.txt (casi ordenado)
Seleccione el archivo a procesar:
[!] Entrada inválida. Debe ingresar un número (1 o 2).
EVIDENCIA 1
-----
1. Cargar log607-1.txt (desordenado)
2. Cargar log607-2.txt (casi ordenado)
Seleccione el archivo a procesar:

```

Caso 1.

#### Caso 2: Búsqueda de Rango Vacío

**Descripción:** Se realizó una búsqueda delimitando un rango de fechas fuera del rango de los registros. Para la fecha de inicio se estableció el año 2030 y para la fecha de fin se solicitó el año 2045.

**Resultado:** La búsqueda binaria retornó “No se encontraron registros dentro del rango de fechas especificado” sin provocar errores de índice fuera de rango (out of bounds). Solicitó al usuario si se desea realizar otra corrida.

```

└─┐Deseas realizar una busqueda por rango de fechas en estos datos ordenados? (s/n): s
-----
Busqueda por rango de fecha
-----
--- Ingrese la Fecha INICIAL ---
Mes (ej. Sep): Oct
Dia (ej. 29): 30
Año (ej. 2024): 2030
Hora (HH:MM:SS, ej. 14:00:00): 14:00:20

--- Ingrese la Fecha FINAL ---
Mes (ej. Oct): Nov
Dia (ej. 05): 10
Año (ej. 2024): 2045
Hora (HH:MM:SS, ej. 18:00:00): 19:45:20

No se encontraron registros dentro del rango de fechas especificado.

└─┐Desea realizar otra corrida para comparar? (s/n): 

```

Caso 2.

### Caso 3: Bordes Extremos de Fechas

**Descripción:** Búsqueda configurada desde el primer registro del archivo hasta el último registro.

#### Resultado

```

EVIDENCIA 1
-----
Busqueda por rango de fecha
-----
--- Ingrese la Fecha INICIAL ---
Mes (ej. Sep): Sep
Dia (ej. 29): 10
Año (ej. 2024): 2024
Hora (HH:MM:SS, ej. 14:00:00): 09:16:12

--- Ingrese la Fecha FINAL ---
Mes (ej. Oct): Sep
Dia (ej. 05): 07
Año (ej. 2024): 2026
Hora (HH:MM:SS, ej. 18:00:00): 23:50:40

Busqueda completada en 8 microsegundos (O(log n))
Registros encontrados en el rango: 6818
Indice inicial: 0 | Indice final: 6817
Registros del rango guardados en 'range607.txt'.

└─┐Deseas visualizar los primeros registros encontrados en consola? (s/n): 

```

```
C:\MTY.TC1031.607.2613.A00843645 > Evidences > Evidence1 > range607.txt
1 Sep 10 2024 09:16:12 10.14.245.197 Suspicious IP Addresses
2 Sep 10 2024 13:50:52 10.14.99.247 Anomalous User Activity
3 Sep 10 2024 18:14:07 10.14.153.89 Anomalous User Activity
4 Sep 10 2024 21:18:36 10.14.183.213 Data Exfiltration Attempts
5 Sep 11 2024 06:44:21 10.14.80.172 Suspicious IP Addresses
6 Sep 11 2024 08:48:35 10.14.45.122 Anomalous User Activity
7 Sep 11 2024 13:23:41 10.14.241.4 Changes in File Integrity
8 Sep 11 2024 17:10:13 10.14.138.230 Encryption Failures
9 Sep 12 2024 00:25:50 10.14.18.96 Human error
10 Sep 12 2024 03:30:41 10.14.52.18 SQL injection
11 Sep 12 2024 03:31:18 10.14.13.16 Changes in File Integrity
12 Sep 12 2024 03:58:23 10.14.125.11 Encryption Failures
13 Sep 12 2024 04:03:41 10.14.79.25 Application error
14 Sep 12 2024 05:48:17 10.14.37.249 Malware Detection Alerts
15 Sep 12 2024 06:35:19 10.14.211.220 Unauthorized Access Attempts
```

Primer fragmento del archivo “range607.txt”

```
C:\MTY.TC1031.607.2613.A00843645 > Evidences > Evidence1 > range607.txt
6803 Sep 6 2026 11:06:40 10.14.114.230 Data Exfiltration Attempts
6804 Sep 6 2026 15:05:17 10.14.0.225 Changes in File Integrity
6805 Sep 6 2026 15:14:46 10.14.134.171 Configuration errors
6806 Sep 6 2026 22:46:22 10.14.15.103 Data corruption
6807 Sep 7 2026 00:01:24 10.14.1.130 Configuration errors
6808 Sep 7 2026 00:35:05 10.14.34.58 Anomalous User Activity
6809 Sep 7 2026 04:21:38 10.14.48.44 Encryption Failures
6810 Sep 7 2026 04:39:58 10.14.43.154 Network connectivity issues
6811 Sep 7 2026 08:19:52 10.14.237.112 Weak or easily guessable passwords
6812 Sep 7 2026 11:42:38 10.14.229.232 Data Exfiltration Attempts
6813 Sep 7 2026 11:52:19 10.14.48.218 Data corruption
6814 Sep 7 2026 14:39:06 10.14.212.167 Network connectivity issues
6815 Sep 7 2026 16:48:14 10.14.119.222 Security breaches
6816 Sep 7 2026 17:30:32 10.14.109.102 Port Scanning Activity
6817 Sep 7 2026 23:08:26 10.14.227.119 Data Exfiltration Attempts
6818 Sep 7 2026 23:50:40 10.14.178.169 Security breaches
6819
```

Último fragmento del archivo “range607.txt”

#### Caso 4: Timestamps Duplicados en log607-2.txt

**Descripción:** Se seleccionó un rango de fecha/hora cuya clave coincide exactamente con múltiples registros duplicados presentes en log607-2.txt.

**Comportamiento:** La función lowerBound localizó el primer elemento (límite izquierdo) y upperBound localizó el último elemento (límite derecho).

**Resultado:** Se incluyeron todos los registros duplicados dentro del intervalo en range607.txt sin perder ni duplicar ninguno.

**Fragmento de archivo generado después del algoritmo de ordenamiento (“output608.txt”)**



## Caso 5: Verificación de Exportación de Archivos

**Verificación de output607.txt:** Se confirmó que el archivo exportado mantenga exactamente el formato de log original (Mes Día Año Hora IP Mensaje) completamente ordenado.

```
C:\MTY.TC1031.607.2613.A00843645 > Evidences > Evidence1 > output608.txt
1 Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
2 Sep 08 2024 04:37:37 10.14.248.79 Configuration errors
3 Sep 08 2024 04:44:15 10.14.135.12 Application error
4 Sep 08 2024 05:50:57 10.14.22.235 Changes in File Integrity
5 Sep 08 2024 10:42:42 10.14.40.195 Data corruption
6 Sep 08 2024 10:47:52 10.14.24.223 Data corruption
7 Sep 08 2024 15:23:16 10.14.206.243 Weak or easily guessable passwords
8 Sep 08 2024 15:25:34 10.14.220.55 Broken authentication and session management
9 Sep 09 2024 01:51:20 10.14.27.106 Application error
10 Sep 09 2024 04:53:57 10.14.96.200 Suspicious IP Addresses
11 Sep 09 2024 07:53:44 10.14.230.164 Data Exfiltration Attempts
12 Sep 09 2024 15:18:08 10.14.182.26 Suspicious IP Addresses
13 Sep 09 2024 18:59:55 10.14.132.80 Failed Login Attempts
14 Sep 09 2024 19:17:57 10.14.186.180 Cross-Site Scripting
15 Sep 10 2024 09:16:12 10.14.245.197 Suspicious IP Addresses
16 Sep 10 2024 13:50:52 10.14.99.247 Anomalous User Activity
17 Sep 10 2024 18:14:07 10.14.153.89 Anomalous User Activity
```

## 4. Conclusión y Recomendaciones

### Hallazgos Principales

#### *Diferencia entre datasets (Desordenado vs. Casi Ordenado):*

Los algoritmos cuadráticos elementales como Insertion Sort y Bubble Sort mostraron una mejora drástica de rendimiento en log607-2.txt (de varios segundos/minutos a milisegundos). Esto ya que en un conjunto casi ordenado la cantidad de desplazamientos e intercambios se reduce a  $O(n)$ .

Por otro lado, Selection Sort y Swap Sort no aprovecharon el preordenamiento de los datos, manteniendo tiempos altos debido a sus bucles anidados fijos  $O(n^2)$ .

#### *Eficiencia de Algoritmos $O(n \log n)$ :*

Quick Sort, Merge Sort y Shell Sort procesaron la totalidad del dataset en menos de 50 milisegundos sin importar el nivel de desorden, demostrando la superioridad de las estrategias de "divide y vencerás" para procesamiento masivo de datos.

#### *Búsqueda por Rango:*

La combinación de ordenar el dataset y aplicar Búsqueda Binaria (lowerBound y upperBound) redujo la localización de subrangos a microsegundos ( $O(\log n)$ ), lo que resulta significativamente más eficiente que realizar una búsqueda secuencial de  $O(n)$ .

### Recomendaciones

Para procesamiento de logs en sistemas reales donde las entradas llegan "casi ordenadas" por desfases leves de reloj, Insertion Sort o Quick Sort representan las opciones más viables.

Se recomienda siempre aplicar una búsqueda basada en lowerBound / upperBound para garantizar la integridad en la selección de rangos cuando existan registros de eventos simultáneos (timestamps idénticos).

